Generate–Verify–Repair for Intent-Driven Network Changes: Controlled Evaluation with a Deterministic Verifier

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We evaluate whether a generate–verify–repair (GVR) pipeline—single-shot LLM generation followed by a deterministic network verifier and up to one automated repair—reduces accepted unsafe network changes versus LLM-only generation on a standardized synthetic NetOps intent workload. In a preregistered paired design (study-hosted-netops-gvr-v1-2026-08-29) we ran N=500 distinct synthetic intents (shared single initial candidate per intent) with generation by gpt-5-mini and adjudication by a deterministic verifier. Results: guarded (GVR) accepted 500/500 final changes; baseline (LLM-only) accepted 496/500 (paired difference = 0.008). The preregistered primary exact conditional McNemar two-sided test on the discordant pairs (n10 = 4, n01 = 0) yields p = 0.125 (computed as p = 2 * PBinom(X >= 4; n = 4, p = 0.5)). An exploratory paired bootstrap percentile 95% CI (10,000 resamples, seed = 1729) = [0.002, 0.016] (bootstrap labeled exploratory per preregistration and interpreted cautiously given only four discordants). Repairs were invoked for 4 initial candidates; total model calls used = 504 (500 shared_initial + 4 repair calls). The preregistered templated-automation comparator was not executed. Because the preregistered decision rule is the exact conditional McNemar (p = 0.125), we do not claim confirmatory statistical significance. We reconcile the conditional McNemar and the exploratory bootstrap CI, document verifier scope and known blind spots, provide execution accounting and representative validator traces, and give explicit artifact pointers and hashes for reproducibility.**

I. Introduction

Operator intent→device configuration translation can reduce manual effort, but errors in generated CLI sequences or transient unsafe states can cause outages. Modern large language models (LLMs) show promise for translating human intent to device configuration, yet hallucination and factual errors remain central reliability concerns for operational NetOps. A practical mitigation architecture is generate–verify–repair (GVR): produce a candidate configuration with an LLM, deterministically verify it against encoded workflow and policy rules, and, if verification fails, run a bounded automated repair attempt. This paper reports a preregistered, artifact-grounded paired experiment that asks: Does a GVR pipeline (LLM → deterministic verifier → up to one automated repair) produce fewer accepted unsafe changes than LLM-only generation on a standardized synthetic NetOps intent workload? We contribute: (1) a preregistered paired trial (N=500 paired intents) executed end-to-end with archived artifacts; (2) an artifact-grounded description of generation, verifier semantics, and the bounded repair loop; (3) reconciled confirmatory inference (two-sided conditional exact McNemar) and exploratory paired bootstrap CI descriptions with an explicit small-discordant-count caveat; and (4) execution accounting, representative validator traces, and a discussion of verifier blind spots and practical external-validity limits.

II. Related Work

Three converging literatures motivate and contextualize this study. First, recent surveys and system papers document active interest in applying LLMs to NetOps tasks such as intent translation, configuration generation, AIOps, and multi-agent orchestration. These works show practical system designs and report deployment experiences or prototypes while consistently highlighting reliability risks from hallucination and factual errors in generated artifacts [doi:10.1145/3718958.3750537; https://openalex.org/W4415071524; doi:10.1002/nem.2313]. They motivate architectures that pair generative models with domain tooling rather than relying on unconstrained generation alone.

Second, methodological literature on tool-augmented LLMs and generate-validate-repair pipelines argues that external deterministic or domain tools (verifiers, simulators, parsers) materially change the failure-mode profile of LLM outputs and therefore should be central to evaluation design. Several studies propose or evaluate RAG/tool-augmented workflows and stress the need for standardized evaluation practices that explicitly measure verifier coverage, oracle mismatch, and tool-call cost tradeoffs [https://openalex.org/W4403443637; doi:10.2139/ssrn.6647800]. This body of work clarifies two practical points relevant to our design: (1) verification tools provide high-precision adjudication for well-specified properties (syntactic correctness, required sequencing, availability constraints) but rarely capture all production hazards (vendor idiosyncrasies, timing/convergence effects); and (2) the operational cost of invoking heavyweight verifiers or iterative repair loops must be measured alongside safety benefits.

Third, the verification and network-analysis literature supplies deterministic or formal checkers capable of validating many configuration properties at scale. Recent verifier and distributed-verification systems adapt Batfish-style analyses and localized sub-specification approaches to large inventories, enabling automated checks of reachability, required sequences, and group/availability constraints [https://openalex.org/W4413757123;

doi:10.1145/3718958.3750516]. These verifiers form the practical oracle in generate–verify workflows but, as prior work emphasizes, their utility depends on the modeled properties and the abstraction fidelity used to simulate device behavior. Our study builds on these foundations by pairing a compact deterministic verifier (explicitly encoding required-sequence, group/availability, paired-field, and final-state checks) with a tightly bounded repair policy and measuring end-to-end outcomes using preregistered paired inference.

Gaps and relation to the present experiment. While prior systems integrate verification into LLM-driven NetOps frameworks, few published controlled, preregistered experiments compare end-to-end GVR pipelines against LLM-only or templated automation baselines at the paired-intent scale with full artifact archival. The methodological calls for standardized evaluation (tool budgets, verifier coverage reporting, and paired designs) directly informed our preregistration. Our contribution is therefore empirical and procedural: a preregistered, paired measurement using a deterministic verifier as the adjudicating oracle and an explicitly bounded single-repair policy, with complete archival of generation outputs, verifier traces, and analysis artifacts to permit independent reconstruction and targeted follow-up studies that address verifier fidelity and production realism.

III. Methodology

Preregistration and confirmatory plan: The experimental design, sampling, estimands, and primary decision rule were preregistered as study-hosted-netops-gvr-v1-2026-08-29. The primary estimand is the paired success-rate difference (GVR - baseline) across N = 500 distinct intents stratified evenly over 10 synthetic topology profiles (50 intents per profile). Sampling seeds, the deterministic task generator, and the analysis executor were frozen prior to execution (preregistration manifest SHA-256 = aa8982a27c73e51521ac023a 78adcfa358ef3978c0f9bc05213801fcbd2f1d0a). The preregistered confirmatory decision rule is the two-sided conditional exact McNemar computed on the observed discordant pairs; the preregistration also specified an exploratory paired bootstrap percentile interval (10,000 resamples; bootstrap_seed = 1729) as a descriptive interval only.

Task generation and paired design: A deterministic task generator (deterministic_netops_task_generator_v5) produced 500 distinct intent tasks (task_manifest entries archived). Each intent was executed under two paired conditions using the same shared_initial_generation candidate: (1) baseline—accept the shared initial LLM candidate as the final change; and (2) guarded (GVR)—validate the shared candidate with the deterministic verifier and, if failing, invoke at most one automated repair attempt. Pairing ensured within-intent control of task difficulty, topology profile, and the single shared initial candidate design as preregistered.

Generation model and orchestration: The declared generation model for the run is gpt-5-mini (declared_model_version recorded in execution/model_configuration.json; execution/model_configuration.json SHA-256 = a40e96e2 12ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd07359 7ffdd820). Per the execution manifest and provider audit, the run produced 500 shared_initial_generation calls and 4 repair calls (hosted_model_call_event_count = 504; model_calls_used = 504). The orchestration adapter family is hosted_netops_gvr_v1 using the hosted provider recorded as 'openai_responses'. The maximum_repair_calls_per_task was set to 1 in the preregistration and enforced by the controller.

Deterministic verifier: role, semantics, and outputs: The primary adjudicator is deterministic_netops_validator_v5 (validator_version = 5.0), implemented as a deterministic canonicalization and rule-checking pipeline. For each candidate CLI sequence, the verifier executes the following stages in order: (1) syntactic parsing and canonical normalization (tokenization of interface commands, canonical ordering of multi-line sequences, removal of formatting variants); (2) command-level required-sequence enforcement (patterns such as must_be_down_for_fields, all_down_before_any_field_change, make_before_break switchover); (3) availability/group constraints (exactly_active, minimum_active_in_groups); (4) dependency and paired-field constraints (paired MTU changes, equal_final_fields between interfaces); and (5) final-state comparison against the task's expected_final_state. Any nonempty violation set yields a failing outcome. For each episode the verifier emits a structured validator_trace containing fields including normalized_configuration, observed_touched_field_count, parsed_command_count, required_command_count, violation_count and list, validation_before/after summaries, and boolean valid. Representative verifier_trace objects are archived under execution/scoring/ (see representative artifact examples). Passing the verifier is a necessary experimental safety gate but is not claimed to be sufficient for all real-world hazards (see Threats to Validity).

Verifier codebook and artifact pointer: The human-readable verifier codebook (violation-code $\rightarrow$ short description) used to construct repair prompts and to interpret validator_trace violations is archived as an artifact referenced by the execution manifest. The execution bundle includes a full hash manifest; the execution manifest path is execution/execution_manifest.json (the manifest enumerates the verifier codebook artifact path and its SHA-256 for direct retrieval). Representative scoring JSON objects in execution/scoring/ contain the violation codes emitted for each failing candidate.

Repair policy, prompt template, and acceptance rule: When the verifier returns valid=false for a shared initial candidate under the guarded condition, the GVR controller issues at most one repair prompt to the same generation model. The repair prompt is constrained and intentionally compact: it contains (a) the original high-level intent abstraction for the task, (b) the verifier violation codes and short descriptions extracted from the verifier_trace, and (c) a request for a corrected canonical CLI sequence that satisfies the listed violation codes. The repair template minimizes additional contextual material to isolate the marginal effect of one repair attempt. A repair candidate is accepted only if a subsequent verifier invocation returns valid=true. All repair prompts, responses, and verifier outcomes are logged and archived in execution/provider_calls/ and execution/scoring/.

Scoring rule and outcome definition: Per preregistration, each intent's final accepted change is scored binary by the primary verifier: success = final accepted configuration passes deterministic_netops_validator_v5 (valid=true) versus unsafe = final accepted configuration fails (valid=false). For baseline pairs the shared candidate is the accepted change without verification-based repair; for guarded pairs acceptance required verifier pass after repair (if any). The analysis used complete pairs only (complete_pair_count = 500) per the preregistered missingness plan; technical failures are recorded and were zero in this run (failed_episode_count = 0).

Statistical procedures and reproducible calculations: The preregistered primary test is the two-sided conditional exact McNemar operating on discordant pairs. Let n10 be the number of pairs where guarded succeeds and baseline fails, and n01 the reverse; n = n10 + n01 and k = n10. The conditional exact two-sided p-value is computed as p = 2 * PBinom(X >= k; n, 0.5), with clipping at 1.0. In this run the archived paired contingency (analysis/paired_contingency_table.csv; archive hash listed in the evidence bundle) is n11 = 496, n10 = 4, n01 = 0, n00 = 0, so n = 4 and k = 4. For n = 4, PBinom(X >= 4; n = 4, p = 0.5) = (1/16), giving p = 2*(1/16) = 0.125. The exact McNemar implementation and this conditional binomial formula were preregistered and applied exactly as documented here; the contingency CSV and analysis logs are archived for reproducibility.

Exploratory paired bootstrap: The preregistration additionally specified an exploratory paired bootstrap percentile CI for descriptive interpretation. We resampled paired outcomes with replacement (10,000 resamples; bootstrap_seed = 1729) and computed the paired success-rate difference (guarded - baseline) per resample; the 2.5th and 97.5th percentiles produce the reported exploratory interval [0.002, 0.016]. The bootstrap was explicitly labeled exploratory in the preregistration because percentile intervals can have poor frequentist coverage when discordant counts are very small and sample margins are near ceiling; we therefore present the bootstrap as a descriptive interval and rely on the preregistered exact McNemar for the primary confirmatory decision.

Provider accounting, determinism, and retry policy: Provider audit fields and the deterministic reconciliation artifact record hosted_model_call_event_count = 504 (completed_hosted_model_call_event_count = 504; failed_hosted_model_call_event_count = 0), stage_counts = {shared_initial_generation: 500, repair: 4}, and model_calls_used = 504. The controller enforced maximum_attempts_per_call = 1 and maximum_repair_calls_per_task = 1 as preregistered. All random seeds used for task generation, bootstrap resampling, and other nondeterministic steps are recorded in the archived artifacts to ensure computational reproducibility.

Predefined exclusion, missingness, and sensitivity: Per preregistration there were no exclusions applied; complete_pair_count = 500 and no technical failures occurred. The preregistration specified a sensitivity analysis treating technical failures as unsafe; because no such failures occurred the primary and sensitivity treatments coincide. All missingness, contamination, and reconciliation artifacts are archived (analysis/missingness_summary.csv, analysis/contamination_summary.csv, deterministic_reconciliation.json).

Reproducibility artifacts and pointers: Key archived artifacts referenced in this methodology include execution/raw_results.jsonl (SHA-256 = fe9b93ef 5cc5a237a3361f18f45fabf14cd23a1c679556f65477f9f6 81915d02), execution/task_manifest.jsonl (SHA-256 = 5a822e787302a0b3bb03a86a81b20564a44e2636731f853f 9d45ac12e4876cc8), analysis/paired_contingency_table.csv (SHA-256 = 8fb135cec541cbe0b14f16125db33154a41460c4 9958427fe8e7f3e1f2abfece), and analysis/analysis_log.jsonl (SHA-256 = 261ec80ea9343fccbcd0a62294f5382773770fd5 9d387dc9d749de50ca909b3e). The full execution manifest (execution/execution_manifest.json) contains the complete list of artifact paths and SHA-256 digests for direct retrieval. Reproducible calculation parameters (bootstrap resamples, bootstrap_seed = 1729) and the exact McNemar formula are recorded in analysis/analysis_log.jsonl.

Implementation notes and limits applied to the methodology: The verifier enforces canonical, task-specified properties and exact final-state equality against the synthetic expected_final_state; it intentionally omits vendor-specific parser idiosyncrasies, control-plane timing/simulation, and out-of-band hardware constraints. The repair policy is intentionally bounded (single repair) to limit model-call overhead and to measure the marginal corrective value of one repair attempt in an operationally conservative budget. These design choices were preregistered and are explicitly documented to bound claims about operational generality.

IV. Results

Execution and primary counts: The run completed completed_episode_count = 1000 (500 intents × 2 conditions) with model_calls_used = 504. analysis/condition_summary.csv (archived hash: f3e197425cc03dec41cda77f078f6d0b 079b06bbde7ac736de5fe988b027ac58) reports baseline successes 496/500 (0.992) and guarded successes 500/500 (1.000). The paired contingency table (analysis/paired_contingency_table.csv; archived hash: 8fb135cec541cbe0b14f16125db33154a41460c49958427f e8e7f3e1f2abfece) is: n11 = 496, n10 = 4, n01 = 0, n00 = 0. Repairs were invoked for four initial candidates; all four repair attempts resulted in subsequent verifier passes and correspond to the four discordant pairs (n10 = 4). Mean model calls per accepted change ≈ 504/500 = 1.008 and median calls per accepted change = 1 (most accepted changes required only the shared_initial_generation). Provider audit (deterministic_reconciliation.json in the evidence bundle) records hosted_model_call_event_count = 504, completed_hosted_model_call_event_count = 504, failed_hosted_model_call_event_count = 0, cache_hit_count = 0, cache_miss_count = 504, and stage_counts: shared_initial_generation = 500, repair = 4.

Confirmatory exact McNemar calculation (preregistered primary decision rule): Observed discordants n = n10 + n01 = 4 with k = n10 = 4. Using the conditional exact binomial formulation described in Methods, p = 2 * PBinom(X >= 4; n = 4, p = 0.5). For n = 4, PBinom(X >= 4; n = 4, p = 0.5) = 1/16, so p = 2 * (1/16) = 0.125. Because this exact conditional test was the preregistered decision rule, we do not claim confirmatory statistical significance for the primary estimand.

Exploratory paired bootstrap interval and its interpretation: The preregistered exploratory paired bootstrap percentile 95% CI (10,000 resamples; seed = 1729; artifacts archived as analysis/paired_difference_figure.svg and analysis/analysis_log.jsonl; analysis_log.jsonl SHA-256 = 261ec80ea9343fccbcd0a62294f5382773770fd59d387dc9 d749de50ca909b3e) for the paired difference is [0.002, 0.016]. This bootstrap is explicitly exploratory in the preregistration. Practically, with near-ceiling margins (496 and 500 successes) and only four discordant pairs, the conditional exact McNemar conditions on the observed margins and treats discordants as a small binomial sample; under that conditioning the two-sided tail probability yields p = 0.125. The bootstrap resamples paired outcomes without conditioning on margins and can therefore produce a nonzero lower CI bound even when the conditional McNemar is non-significant. Small discordant counts impair the frequentist coverage properties of percentile bootstrap intervals; we therefore present the bootstrap interval as a descriptive, exploratory summary rather than as a second confirmatory decision. This distinction follows the preregistered analysis_plan and multiplicity handling.

Repair diagnostics and representative artifacts: The four repaired episodes were flagged by the verifier for verifier-detectable sequencing or availability violations (e.g., missing make_before_break availability ordering or required must_be_down_for_fields violations). Each repaired episode received a single constrained repair prompt that included the intent abstraction and the verifier violation codes; the repair candidate accepted by the verifier corrected the flagged sequence or availability issue and returned valid=true. Representative verifier_trace JSON entries and their fields (validation_before, validation_after, violation_count, normalized_configuration) are archived under execution/scoring/ for inspection. Example representative scoring JSONs (archived artifacts included in the evidence bundle) include: - execution/scoring/task-000001-baseline.json (SHA-256 = e97212b62733ba30de344b8980d06b59cfecb2d7 91daa91332b3d617a52f45f8) — shows validator_version = 5.0 and a completed valid=true outcome for the shared candidate; the file includes validation_before/after blocks and the normalized_configuration. - execution/scoring/task-000002-baseline.json (SHA-256 = 2d458e3e08306521 5264fa509e0df3fed38cf160881f7e6a143c4ddd83034b92) — covers a 'make_before_break_uplink_switchover' hard pattern. These representative artifacts illustrate the verifier emitted fields and normalization semantics; full per-episode raw results and scoring JSONs are listed in the execution manifest and raw_results.jsonl (execution/raw_results.jsonl SHA-256 = fe9b93ef5cc5a237a3361f18f45fabf14cd23a1c 679556f65477f9f681915d02).

Contingency accounting and provider audit consistency: The deterministic_reconciliation.json produced by the analysis executor confirms accounting consistency: task_count = 500 (paired intents), planned_episode_count = 1000, completed_episode_count = 1000, complete_pair_count = 500, baseline_success_count = 496, guarded_success_count = 500, and contingency entries n_11 = 496, n_10 = 4, n_01 = 0, n_00 = 0. Provider_call_audit in the reconciliation artifact matches the execution manifest stage_counts and the model_calls_used summary. These machine-verifiable reconciliation artifacts and their hashes are archived in the evidence bundle for independent reproduction of the contingency table and the McNemar calculation.

Operational interpretation and cost implication: In this synthetic workload the bounded GVR controller intercepted four verifier-detectable hazards at low LLM overhead (repair rate 0.8%, four additional model calls). Mean model-call overhead per accepted change was ~0.008 extra calls beyond the single shared initial generation. These observations show that a compact verifier plus a bounded repair loop can remove verifier-detectable unsafe outputs cheaply in this controlled setting; scaling to production requires measuring verifier runtime, expanding verifier fidelity (vendor parsing, timing/convergence models), and stress testing with adversarial NetOps inputs where repair frequency and cost may increase.

Synthesis relative to preregistered power assumptions: The preregistered power plan targeted an absolute paired increase ≈ 0.08. The observed baseline success rate (496/500 = 0.992) left little headroom and produced only four discordant pairs, substantially reducing realized confirmatory sensitivity relative to that plan. This ceiling effect explains why an empirically small paired difference (0.008) yields an exploratory bootstrap CI excluding zero while the preregistered conditional exact McNemar is non-significant; both results are reported and interpreted according to the preregistered analysis_plan.

V. Discussion

Interpretation and reconciliation of inferential outcomes: The GVR pipeline prevented four verifier-detectable unsafe initial candidates that baseline would have accepted. Under the preregistered conditional exact McNemar (p = 0.125) this difference is not confirmatory. The exploratory bootstrap CI excludes zero but should be interpreted cautiously: the conditional McNemar conditions on observed margins and evaluates the binomial tail of discordant outcomes; with n = 4 and k = 4 this yields p = 0.125. The bootstrap resamples pairs without fixing margins and can produce a nonzero lower CI bound when most pairs are concordant and a small number are discordant. Because the bootstrap was predeclared exploratory and because small discordant counts reduce frequentist coverage guarantees for percentile intervals, we report both inferential perspectives and follow the preregistered decision rule for the primary claim.

Exact McNemar implementation and reproducibility note: For clarity and reproducibility we spell out the exact conditional McNemar computation used in the preregistration and analysis. From the archived paired contingency (n11=496, n10=4, n01=0, n00=0) we set n = n10 + n01 = 4 and k = n10 = 4. The preregistered two-sided p-value is computed via the conditional binomial on discordant pairs as p = 2 * PBinom(X >= k; n, 0.5), with PBinom denoting the cumulative tail probability of a Binomial(n, 0.5). For n = 4 and k = 4, PBinom(X >= 4; 4, 0.5) = (1/2)^4 = 1/16, so p = 2*(1/16) = 0.125. The archived paired contingency CSV (analysis/paired_contingency_table.csv; SHA-256 = 8fb135ce c541cbe0b14f16125db33154a41460c49958427fe8e7f3e1 f2abfece) and the analysis log (analysis/analysis_log.jsonl; SHA-256 = 261ec80ea9343fccbcd0a62294f5382773770fd5 9d387dc9d749de50ca909b3e) permit exact reproduction of the reported p-value and of the bootstrap procedure (bootstrap_seed = 1729, resamples = 10,000).

Why a bootstrap CI can exclude zero when the exact McNemar is non-significant: The two methods answer related but distinct inferential questions. The conditional exact McNemar conditions on the observed total discordant count n and tests symmetry of discordant outcomes under the null; it is exact for small n and is the preregistered decision rule here. The paired bootstrap resamples paired outcomes (preserving pair identity but not fixing marginal counts) and produces a percentile interval for the paired success-rate difference; because bootstrap resampling treats the observed concordant majority as informative about variability, the resulting percentile CI can exclude zero even when the conditional test (which focuses solely on the small discordant mass) yields a non-significant p. With near-ceiling margins (baseline $\approx$ 0.992) and very few discordants (n = 4) the bootstrap interval's frequentist coverage properties are fragile; this is why the bootstrap result is explicitly labeled exploratory in the preregistration and in our reporting.

Verifier coverage, blind spots, and operational sufficiency: The deterministic verifier enforces canonical normalization, required sequencing, availability/group constraints, dependency and paired-field rules, and exact final-state equality to the task expected_final_state. These checks are necessary for detecting a broad class of syntactic and workflow errors that LLMs commonly produce for our intent grammar, and they were sufficient to flag the four repaired episodes in this run. However, the verifier does not model vendor-specific CLI semantics, control-plane timing and convergence effects, session or execution-time errors, hardware resource limits, or subtle environment side effects; passing the verifier is therefore necessary for experiment safety but not sufficient to guarantee production safety. Expanding verifier fidelity (vendor parsers, control-plane simulators, timing/convergence models) is required before operational automation.

Artifact accessibility and the verifier codebook: For artifact-grounded inspection, the full execution manifest enumerates the verifier codebook (violation-code $\rightarrow$ human-readable description) and the artifact path plus SHA-256 for the codebook. The execution manifest is archived as execution/execution_manifest.json in the evidence bundle and lists the exact verifier codebook artifact path and its SHA-256 for direct retrieval; users reproducing or auditing this study should begin from the archived execution manifest to retrieve the codebook and the associated verifier-trace JSON objects under execution/scoring/.

Operational implications and cost tradeoffs: A compact deterministic verifier plus a bounded repair loop intercepted verifier-detectable hazards at low LLM overhead in this synthetic workload. Observed repair rate was 4/500 (0.8%), implying modest additional model calls under the bounded policy; provider accounting records hosted_model_call_event_count = 504 with stage_counts: shared_initial_generation = 500 and repair = 4. In operations this pattern suggests that a low-latency verifier integrated as a gate can materially reduce verifier-detectable unsafe proposals at modest model-call cost when baseline failure rates are low. Real deployments must measure verifier runtime on full inventories, evaluate how repair frequency scales under adversarial or more diverse intent distributions, and balance the latency/cost of higher-fidelity verifiers against residual operational risk.

Threats to validity and generality (expanded diagnostics): Internal validity is strengthened by preregistration, fixed seeds, and complete accounting (complete_pair_count = 500; no failed episodes). Construct validity is limited by the verifier's abstraction level and by the synthetic device templates (synthetic_topologies_v1 and synthetic_device_configs_v1): the verifier enforces encoded workflow and group constraints that match the synthetic intent grammar, but those rules are an imperfect model of vendor behavior and control-plane dynamics. External validity is constrained by single-model scope (gpt-5-mini) and synthetic topologies; multi-model replication, vendor-specific parsing, and live telemetry tests are required to generalize. Conclusion validity was affected by realized margins: the preregistered power plan assumed a larger baseline failure rate and targeted an absolute paired increase $\approx$ 0.08; the observed near-ceiling baseline (496/500 = 0.992) yielded only four discordant pairs and reduced realized power relative to the preregistered assumption. These limitations are documented in the archived manifest and preregistration, and they explain why a modest numerical paired difference appears in the exploratory bootstrap interval but not in the preregistered exact McNemar decision.

VI. Limitations

- Synthetic tasks and topology profiles were used (synthetic_topologies_v1 and synthetic_device_configs_v1); external validity to production hardware, vendor-specific

CLIs, live telemetry, and control-plane timing effects is untested.
- Only a single generation model (gpt-5-mini) was evaluated; multi-model generality and replication remain to be established.
- Safety in this experiment is defined relative to deterministic_netops_validator_v5's encoded rule set (syntactic parsing/normalization, required-sequence/workflow-policy checks, protected-interface rules, group and availability constraints, dependency-rule enforcement, and final-state comparison to the task expected_final_state). Passing this verifier is necessary for the experiment's outcome but not sufficient for all real-world safety properties.
- The preregistered power plan targeted an absolute paired increase ≈ 0.08 (8 percentage points). The observed baseline success rate (496/500 = 0.992) produced only four discordant pairs, substantially reducing realized power relative to the preregistered assumption and limiting confirmatory sensitivity.
- The preregistered templated-automation comparator (secondary confirmatory arm) was not executed; therefore the preregistered multiplicity plan (two confirmatory tests with Bonferroni correction) could not be fully applied and the familywise error interpretation is partial.

VII. Conclusion

In a preregistered paired trial (N = 500) using gpt-5-mini and a deterministic verifier, a generate–verify–repair pipeline repaired four verifier-detectable unsafe initial candidates at modest model-call cost (4 repairs; model_calls_used = 504). The paired difference = 0.008 yields an exploratory bootstrap 95% CI [0.002, 0.016], while the preregistered exact conditional McNemar test (n10 = 4, n01 = 0) yields p = 0.125; under the preregistered decision rule we do not claim confirmatory significance. Deterministic verification plus a bounded repair loop can remove verifier-detectable unsafe outputs with low overhead in synthetic settings; broader operational claims require expanded verifier fidelity, adversarial NetOps evaluation, multi-model replication, and execution of the preregistered templated comparator (which was not executed in this run). All primary execution and analysis artifacts cited here are archived in the evidence bundle for independent verification; see the archived execution manifest for full artifact paths and hashes.

Disclosure Statement

This study was executed autonomously after an explicit autonomy lock. Generation model used in the archived run: gpt-5-mini (declared_model_version recorded in execution/model_configuration.json; execution/model_configuration.json SHA-256 = a40e96e212ab87da 251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820).
Orchestration adapter family: hosted_netops_gvr_v1 using the hosted provider recorded as 'openai_responses'. Key automated components recorded in the run: deterministic_netops_task_generator_v5 (task synthesis), deterministic_netops_validator_v5 (primary deterministic verifier/scorer), and the GVR controller implementing shared_initial_generation plus an automated single-repair step (maximum_repair_calls_per_task = 1). Preregistration: study-hosted-netops-gvr-v1-2026-08-29 (preregistration SHA-256 = aa8982a27c73e51521ac023a78adcfa358ef3978c0f9bc05 213801fcbd2f1d0a). The immutable initial master prompt is archived at provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5a cdf1582bb39b15ba). Primary high-level execution quantities reported here (N = 500 complete pairs; model_calls_used = 504; repair calls = 4; paired counts n11 = 496, n10 = 4, n01 = 0, n00 = 0) are derived from archived execution and analysis artifacts. Representative artifact hashes included in the evidence bundle: execution/raw_results.jsonl SHA-256 = fe9b93ef5cc5a237a3361f18f45fabf14cd23a1c679556f6 5477f9f681915d02; execution/task_manifest.jsonl SHA-256 = 5a822e787302a0b3bb03a86a81b20564a44e2636731f853f 9d45ac12e4876cc8; analysis/paired_contingency_table.csv SHA-256 = 8fb135cec541cbe0b14f16125db33154a41460c4 9958427fe8e7f3e1f2abfece; analysis/analysis_log.jsonl SHA-256 = 261ec80ea9343fccbcd0a62294f5382773770fd5 9d387dc9d749de50ca909b3e. The human-readable verifier codebook (violation-code → description) and the full list of artifact paths and hashes are enumerated in the archived execution manifest (execution/execution_manifest.json); the execution manifest lists the exact verifier codebook artifact path and its SHA-256 for direct retrieval. The design, preregistration, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revision were performed autonomously after the autonomy lock; no human manually edited or rewrote manuscript technical content after that lock. The preregistered templated-automation comparator was not executed in this archived run; that unexecuted arm means the originally preregistered two-test Bonferroni familywise plan could not be fully applied.

References

[1] doi:10.1145/3718958.3750537
[2] Sebastian Simon, Alina Mailach, Johannes Dorn, Norbert Siegmund, "A Methodology for Evaluating RAG Systems: A Case Study On Configuration Dependency Validation", 2024, 10.48550/arxiv.2410.08801
[3] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
[4] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635
[5] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, Jian Yang, Ying Zhang, "Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework", 2025, 10.1145/3718958.3750537